

CryptoLint AI: An LLM-Augmented Static Analysis Tool for Cryptographic Vulnerability Detection in Android Applications

Noha Moussaddak - Hajar Chatbaoui - Kenza Madili - Sana Asseknour^a,

^a*National School of Applied Sciences (ENSA), Marrakech, Morocco, Supervised by Pr.Lachgar Mohammed.*

Abstract

CryptoLint AI (v1.0.0, MIT License) is an open-source, immediately deployable security auditing pipeline that detects cryptographic vulnerabilities and insecure coding practices in Android applications. The software contribution consists of three reusable artefacts: (1) a versioned catalogue of sixteen cryptographic detection rules mapped to NIST SP 800-131A, OWASP MASVS v2, and CWE identifiers, exposed as a queryable REST endpoint; (2) a Python/Flask REST API backend that accepts Android APKs, Java, or Kotlin source files without requiring project build systems, decompiles APKs on-the-fly using Jadx, and returns structured JSON findings; and (3) a self-contained HTML/JavaScript frontend requiring no build toolchain. The tool can be installed in one step (`pip install -r requirements.txt && python app.py`) and validated immediately against the OWASP UnCrackable Level 1 and AndroGoat reproducibility APKs included in the repository. Static findings are augmented by an LLM reasoning layer invoking Qwen/Qwen2.5-Coder-32B-Instruct via the Featherless AI OpenAI-compatible inference endpoint (`temperature=0.1, max_tokens=1024`). The model is constrained to a strict JSON output schema validated at runtime; if schema validation or the API call fails, the tool automatically falls back to template-based remediation advice drawn from each rule's embedded metadata, maintaining full functionality in air-gapped or offline environments. When LLM features are enabled, up to 3,000 characters of the analyzed source code are transmitted to the external Featherless AI endpoint; users analyzing proprietary code should configure `FEATHERLESS_API_KEY` to an empty string to operate in offline-only mode. Validation on the OWASP UnCrackable Level 1 and AndroGoat benchmark APKs confirmed the correct detection of all in-scope vulnerability patterns with no false negatives for the sixteen targeted rules. The source code, reproducibility dataset, and installation instructions are available at <https://github.com/itsnoha0x/CryptoLint-AI>.

Keywords: Android security, static analysis, cryptographic vulnerabilities, LLM-assisted remediation, SAST, mobile application security, OWASP Mobile, MASVS

Required Metadata

C1	Current code version	v1.0.0
C2	Permanent link to code/repository	https://github.com/itsnoha0x/CryptoLint-AI/
C3	Permanent link to reproducible capsule (archived release)	https://doi.org/10.5281/zenodo.20357689
C4	Legal code license	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Python 3.10+, Flask 3.0, JavaScript/HTML/CSS (frontend), Jadx 1.5.0 (APK decompiler), Featherless AI API (Qwen/Qwen2.5-Coder-32B-Instruct)
C7	Compilation requirements, operating environments & dependencies	Python 3.10+, Java JDK 11+ (required by Jadx); pip packages: <code>flask</code> , <code>flask-cors</code> , <code>requests</code> , <code>python-dotenv</code> ; supported on Windows 10+, macOS 12+, and Ubuntu 22.04+; no compilation step required (interpreted Python)
C8	If available: link to developer documentation/manual	https://github.com/itsnoha0x/CryptoLint-AI/blob/main/README.md
C9	Support email for questions	hello@nohawrites.com

Table 1: Code metadata

Nr.	Code metadata description	Details
Nr.	Software metadata description	Details
S1	Current software version	v1.0.0
S2	Permanent link to executables of this version	https://github.com/itsnoha0x/CryptoLint-AI/releases/tag/v1.0.0 (source release; no compiled binary required — runs as a Python script)
S3	Legal software license	MIT License
S4	Computing platform / Operating System	Linux (Ubuntu 22.04+), macOS 12+, Windows 10+; Python 3.10+ runtime required; Java JDK 11+ required for APK decompilation via Jadx
S5	Installation requirements & dependencies	<code>pip install -r requirements.txt</code> installs all Python dependencies (<code>flask</code> , <code>flask-cors</code> , <code>requests</code> , <code>python-dotenv</code>); Jadx 1.5.0 must be available on <code>PATH</code> or configured via <code>JADX_PATH</code> ; optional: <code>FEATHERLESS_API_KEY</code> environment variable for LLM features (tool operates in offline mode without it)
S6	If available: link to user manual	https://github.com/itsnoha0x/CryptoLint-AI/blob/main/README.md — includes quickstart, API reference, configuration options, and CI/CD integration guide
S7	Support email for questions	hello@nohawrites.com

Table 2: Executable software metadata

1. Motivation and Significance

The widespread adoption of Android applications has dramatically expanded the attack surface for cryptographic vulnerabilities. Android applications rely heavily on Java cryptographic APIs to protect sensitive user data. Yet, multiple empirical studies consistently demonstrate that a large proportion of real-world apps implement these APIs incorrectly [1, 2]. Common failures include the use of deprecated hash functions such as MD5 and SHA-1, insecure cipher modes (most prevalently AES in ECB mode, which is the JVM default when no mode is specified), static initialization vectors (IVs) that render encryption deterministic, hardcoded cryptographic keys extractable

through bytecode reverse engineering, weak pseudo-random number generators unsuitable for cryptographic use, and improperly configured TLS connections. All of these weaknesses are straightforwardly exploitable against pre-compiled Android APKs, which are reversible using freely available tools such as Jadx [7] or Apktool.

These vulnerabilities map directly to the OWASP Mobile Top 10 [3], particularly to M1 (Improper Credential Usage), M2 (Inadequate Supply Chain Security), M3 (Insecure Authentication and Authorization), and M5 (Insecure Communication). They are codified in CWE entries CWE-295 through CWE-338, addressed in NIST SP 800-131A [5] and NIST SP 800-38A/D [6], and mapped in OWASP MASVS v2 controls MSTG-CRYPTO-1 through MSTG-CRYPTO-6 [4]. These weaknesses are a documented root cause of non-compliance with PCI-DSS and GDPR data protection requirements in mobile applications [1].

Existing static analysis tools either operate at a broad level — flagging many categories of Android vulnerabilities without specializing in the cryptographic layer — or require significant setup and security expertise to interpret findings. MobSF v3.9.7 [8] provides general Android security analysis but does not combine semantic LLM reasoning with fine-grained cryptographic rule matching, nor does it generate AI-assisted code patches inline. Research prototypes such as CryptoGuard [9] and CogniCrypt [10] bring formal dataflow-based analysis of cryptographic API misuse, but impose steep setup requirements — including full Java compilation pipelines and IDE integration — making them inaccessible for rapid auditing of pre-compiled APKs without project source availability.

CryptoLint AI addresses this gap by providing a reusable, immediately deployable software pipeline comprising: (1) a REST API backend accepting raw APKs, Java, or Kotlin source files without requiring project build systems or source code availability; (2) a versioned rule catalogue of sixteen cryptographic detection rules grounded in NIST and OWASP/MASVS standards, designed for extension and independent reuse; (3) an LLM enrichment layer providing risk scoring, attack scenario narratives, and compliance impact assessment per analysis session; (4) a per-finding patch generation endpoint producing copy-paste-ready Java remediation code; and (5) a structured JSON output format suitable for downstream consumption by CI/CD pipelines, security dashboards, and external tooling.

2. Software Description

2.1. Software Architecture

CryptoLint AI follows a two-layer architecture: a static analysis engine that operates deterministically on source code or decompiled bytecode, and an LLM reasoning layer that provides contextual risk assessment and remediation generation. Both layers are exposed through a Flask REST API backend consumed by a lightweight HTML/JavaScript frontend. Figure 1 presents the full component-level architecture of the system.

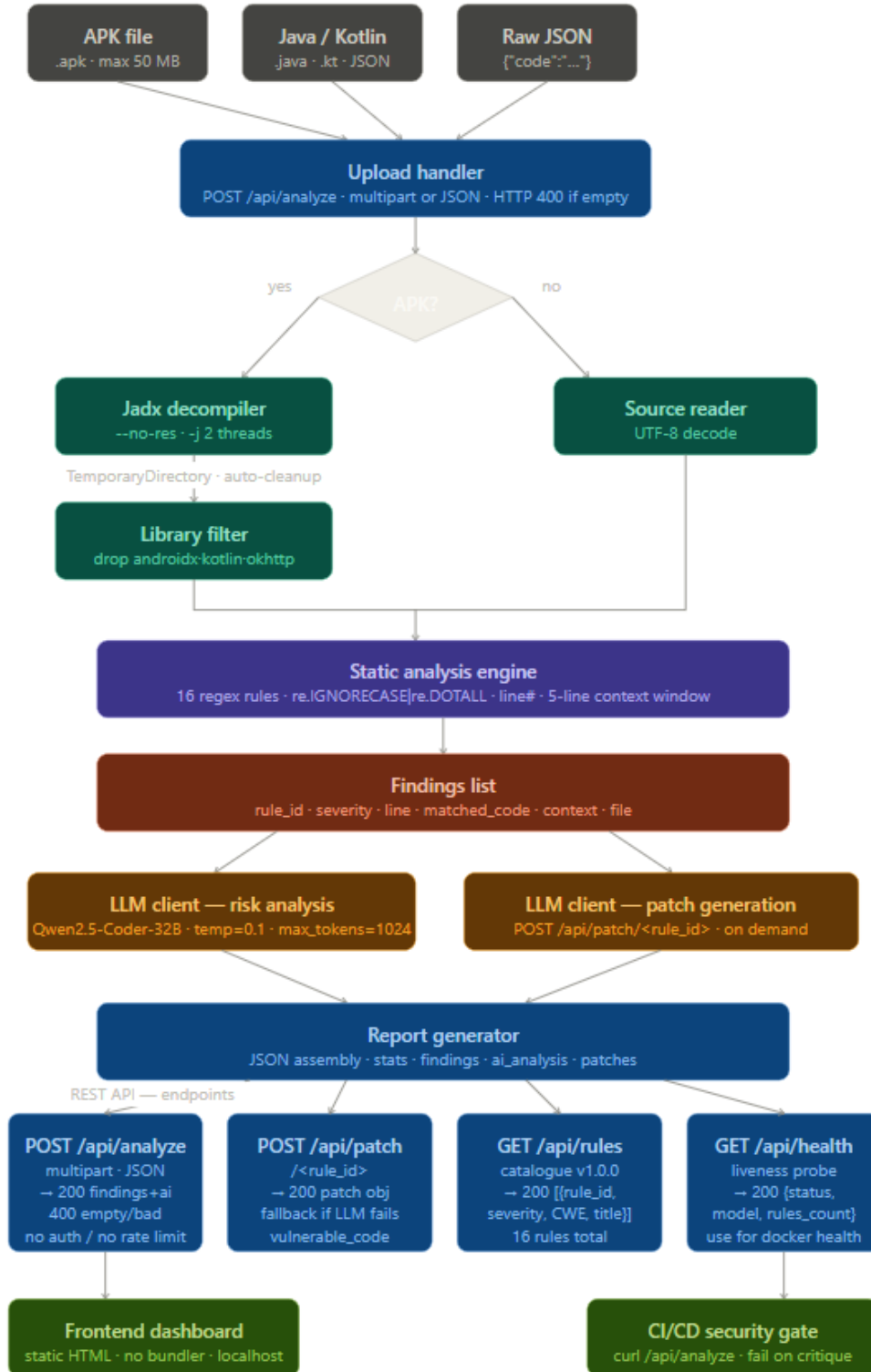


Figure 1: CryptoLint AI component-level architecture. The upload handler routes input files to either the Jadx decompilation path (APK) or the direct source reader (Java/Kotlin). The static analysis engine applies sixteen cryptographic detection rules; findings are forwarded to the Featherless AI LLM client for risk reasoning and patch generation. The report generator assembles the structured JSON response consumed by the frontend dashboard and optionally by CI/CD pipelines.

Deployment and runtime.. The tool runs as a local Flask development server (`python app.py`, default port 5000) or can be containerized. Runtime configuration is managed exclusively through environment variables loaded from a `.env` file at startup: `FEATHERLESS_API_KEY` (required for LLM features), `JADX_PATH` (optional override for the Jadx binary path), and `JAVA_HOME` (optional JVM path override). APK decompilation uses Python’s `tempfile.TemporaryDirectory` context manager, guaranteeing automatic cleanup of all temporary files on process exit or crash, regardless of whether decompilation succeeds or fails partially. Submitted APKs are not persisted to disk beyond the analysis session. File-size limits and authentication are left to the deploying environment (reverse proxy or API gateway); the current implementation does not enforce rate limiting natively.

Data handling and privacy note.. When LLM features are enabled, up to 3,000 characters of the analyzed source code are transmitted to the Featherless AI external inference endpoint. Users analyzing proprietary or sensitive source code should either (a) disable LLM features by leaving `FEATHERLESS_API_KEY` unset (the tool remains fully functional for static detection with template-based remediation), or (b) deploy a local inference server exposing an OpenAI-compatible endpoint and configure `FEATHERLESS_BASE_URL` accordingly. No source code is retained by the tool after the response is returned.

Backend REST endpoints.. The backend exposes four REST endpoints, summarized below with their accepted inputs, response schemas, and principal error codes.

- **POST /api/analyze** — accepts `multipart/form-data` with a `file` field (APK, `.java`, or `.kt`), or `application/json` with `{"code": "...", "filename": "..."}.` Returns HTTP 200 with the JSON schema shown in Listing 1, or HTTP 400 if the input is missing, empty, or unparseable.
- **POST /api/patch/<rule_id>** — accepts `application/json` with a finding object (as returned by `/api/analyze`). Returns HTTP 200 with a patch object containing `vulnerable_code`, `patched_code`, `explanation`, and `libs_needed`. Falls back to the rule’s static template if the LLM call fails.
- **GET /api/rules** — returns HTTP 200 with the full rule catalogue as a JSON array (rule ID, severity, category, title, CWE reference, OWASP reference).

- GET `/api/health` — returns HTTP 200 with service status, active model name, and rule count. Used for liveness probing in deployment environments.

Listing 1: Abbreviated JSON response schema returned by POST `/api/analyze`. All fields are present regardless of whether LLM features are enabled; `ai_analysis` fields fall back to static templates when the API key is unset.

```

1 {
2   "success": true,
3   "filename": "app-release.apk",
4   "stats": {
5     "total": 4,
6     "critique": 2,
7     "majeur": 1,
8     "mineur": 1,
9     "categories": ["Weak hash algorithm", "Insecure cipher
10                   mode",
11                   "Hardcoded cryptographic key"],
12     "files_analyzed": ["MainActivity.java"],
13     "lines_analyzed": 312
14   },
15   "findings": [
16     {
17       "rule_id": "AES_ECB",
18       "severity": "critique",
19       "category": "Insecure cipher mode",
20       "title": "AES without mode specified (ECB default)",
21       "description": "...",
22       "fix": "Cipher.getInstance(\"AES/GCM/NoPadding\")",
23       "reference": "CWE-327, NIST SP 800-38A",
24       "file": "MainActivity.java",
25       "line": 42,
26       "matched_code": "Cipher.getInstance(\"AES\")",
27       "context": "41: SecretKeySpec key = ...\n42: Cipher c =
28                   Cipher.getInstance(\"AES\");\n43: c.init(...)"
29     }
30   ],
31   "ai_analysis": {
32     "global_risk_score": 95,
33     "global_summary": "...",
34     "attack_scenarios": [...],
35     "remediation_priority": "...",
36     "compliance_impact": "...",
37     "estimated_exploit_difficulty": "facile",
38     "ai_source": "featherless_ai"
39   }
40 }

```


2.2. Software Functionalities

Cryptographic Rule Engine. The core detection mechanism consists of sixteen hand-crafted regular expression rules applied via Python’s `re` module with flags `re.IGNORECASE` | `re.DOTALL`. Rules are maintained as a versioned list in `app.py` (catalogue version 1.0.0) and are organized into seven vulnerability categories, each mapped to NIST SP 800-131A, OWASP Mobile Top 10, MASVS v2 controls, MASTG test identifiers, and CWE identifiers. Table 3 presents the complete rule catalogue.

For each regex match, the engine records: the matched code snippet (capped at 200 characters to prevent memory issues on large files), the 1-based line number computed by counting newline characters before the match offset, and a five-line context window (two lines before and after the match line). Duplicate matches of the same rule on the same line are retained as separate findings if they originate from distinct match objects (i.e., `re.finditer` is used, not `re.search`). Third-party library packages are excluded by path-prefix matching during the APK decompilation walk: directories whose normalized path contains any of `/androidx`, `/android/`, `/kotlin`, `/google`, `/okhttp`, or `/okio` are skipped entirely.

Severity levels follow a three-tier scale aligned with CVSS v3.1 base-score ranges and MASVS risk levels: *critique* (CVSS ≥ 9.0 / MASVS L2; directly exploitable with public tooling, leading to key recovery or full data compromise), *majeur* (CVSS 7.0–8.9 / MASVS L1; significant risk under most practical threat models), and *mineur* (CVSS < 7.0 / MASVS L1 best practice; context-dependent risk requiring additional conditions to be exploitable).

Known false-positive sources include: (a) algorithm name strings appearing in comments or string literals not passed to a cryptographic API (e.g., a log message containing the word "AES"); (b) library code surviving the path-prefix filter due to non-standard package hierarchies; (c) generated or obfuscated code where method names collide with rule patterns. Known false-negative sources include: (a) algorithm parameters constructed dynamically (e.g., `Cipher.getInstance(getAlgorithm())`), which evade all string-literal rules; (b) Kotlin-idiomatic wrapper APIs that do not surface the underlying Java crypto call textually; (c) Smali bytecode patterns in APKs for which Jadx decompilation fails.

Rule ID	Category	Sev.	CWE	MASVS / MASTG
HASH_MD5	Weak hash algorithm	Critique	CWE-328	MSTG-CRYPTO-4
HASH_SHA1	Weak hash algorithm	Critique	CWE-328	MSTG-CRYPTO-4
AES_ECB	Insecure cipher mode (ECB default)	Critique	CWE-327	MSTG-CRYPTO-2
AES_CBC_NOAUTH	Unauthenticated cipher mode	Majeur	CWE-327	MSTG-CRYPTO-2
DES_USAGE	Deprecated algorithm (DES/3DES)	Critique	CWE-326	MSTG-CRYPTO-4
HARDCODED_KEY	Hardcoded cryptographic key	Critique	CWE-321	MSTG-CRYPTO-1
STATIC_IV	Static or predictable IV/nonce	Critique	CWE-330	MSTG-CRYPTO-3
WEAK_RNG_RANDOM	Weak PRNG (<code>java.util.Random</code>)	Majeur	CWE-338	MSTG-CRYPTO-6
MATH_RANDOM	Weak PRNG (<code>Math.random()</code>)	Majeur	CWE-338	MSTG-CRYPTO-6
SHAREDPREFS_KEY	Insecure key storage	Critique	CWE-312	MSTG-STORAGE-1
INTERNAL_STORAGE_KEY	World-readable key file	Majeur	CWE-732	MSTG-STORAGE-2
SSL_ALL_HOSTS	TLS hostname verification disabled	Critique	CWE-297	MSTG-NETWORK-3
TRUST_ALL_CERTS	TrustManager accepting all certificates	Critique	CWE-295	MSTG-NETWORK-3
HTTP_CLEAR_TEXT	Cleartext HTTP traffic (non-local)	Majeur	CWE-319	MSTG-NETWORK-2
PKCS5_PADDING	Non-AEAD padding risk	Mineur	CWE-649	MSTG-CRYPTO-2
RSA_SMALL_KEY	RSA key size < 2048 bits	Majeur	CWE-326	MSTG-CRYPTO-4

Table 3: CryptoLint AI versioned detection rule catalogue v1.0.0: 16 rules across 7 vulnerability categories, each mapped to CWE identifiers and OWASP MASVS v2 / MASTG controls. Severity levels are aligned with CVSS v3.1 base-score ranges.

APK Decompile Pipeline.. CryptoLint AI handles pre-compiled Android APKs without requiring access to project source code or build systems. When an APK is submitted, the backend invokes Jadx inside a `tempfile.TemporaryDirectory` context manager (ensuring automatic cleanup on process exit or crash), using flags `-no-res` (skip resources) and `-j 2` (two parallel decompilation threads). Jadx is invoked with a carefully managed `PATH` environment that prepends `JAVA_HOME/bin`, preventing resolution conflicts with Node.js-installed tools that can shadow the JVM binary. If Jadx returns a non-zero exit code (indicating partial decompilation), analysis continues on whichever Java files were produced, with a warning logged to stderr. If no Java files are produced, the endpoint returns HTTP 400 with a descriptive error message.

LLM Risk Reasoning.. After the static engine collects findings, CryptoLint AI sends a structured prompt to the Featherless AI API (<https://api.featherless.ai/v1>, OpenAI-compatible endpoint) invoking model `Qwen/Qwen2.5-Coder-32B-Instruct` with `temperature=0.1` and `max_tokens=1024`. The prompt (version 1.0, embedded in `app.py`) includes a severity-annotated summary of up to ten findings and up to 3,000 characters of analyzed source code. The model is instructed to respond exclusively in a strict JSON schema (validated via `json.loads` after stripping any residual markdown fences); if parsing fails, the fallback static response is returned immediately without retry. The prompt instructs the model to provide:

- `global_risk_score`: integer from 0 to 100;
- `global_summary`: 2–3 sentence executive summary;
- `attack_scenarios`: list of concrete exploitation narratives;
- `remediation_priority`: ordered guidance by exploitability;
- `compliance_impact`: explicit mapping to OWASP, GDPR, PCI-DSS;
- `estimated_exploit_difficulty`: *facile*, *moyen*, or *difficile*.

A fallback mechanism ensures the tool remains fully operational when the LLM API is unavailable or the API key is unset: all static findings are returned with templated remediation advice drawn from each rule’s embedded metadata, maintaining usability in air-gapped or offline environments.

AI-Assisted Patch Generation.. For any individual finding, the `/api/patch/<rule_id>` endpoint invokes the LLM to generate a fully annotated Java remediation patch. The model returns four structured fields: the simplified vulnerable code excerpt, the complete corrected code with inline explanatory comments, a concise technical justification, and any required Gradle dependency additions (e.g., `androidx.security:security-crypto:1.1.0-alpha06` for `EncryptedSharedPreferences`). Generated patches target Android API level 23+ (minimum `AndroidKeyStore` availability) and are annotated with the API-level constraint. Patches are intended as remediation guidance and require human review and compilation testing before deployment; they are not guaranteed to compile without modification in all project configurations.

Frontend Dashboard.. A self-contained HTML/CSS/JavaScript frontend provides a drag-and-drop file upload interface, an interactive findings list grouped and color-coded by severity (*critique* / *majeur* / *mineur*), a global risk score display with LLM executive summary and attack scenario narratives, and an on-demand *Generate Patch* button per finding. The frontend requires no build tools, no Node.js, and no bundler. Installation is as follows: (1) start the Flask backend (`python app.py`); (2) open `frontend/index.html` directly in any modern browser (Chrome, Firefox, Edge); (3) the frontend communicates with the backend at `http://localhost:5000` by default. Screenshots of the upload interface and results dashboard are provided in Section 3.

2.3. Sample Code Snippet Analysis

Listing 2 shows a representative Java code fragment containing three vulnerabilities detectable by CryptoLint AI: MD5 hashing, AES in ECB mode, and a hardcoded key string.

Listing 2: Vulnerable Android Java fragment containing three CryptoLint AI detectable weaknesses: HASH_MD5, AES_ECB, and HARDCODED_KEY.

```
1 // [HASH_MD5] MD5 is cryptographically broken since 2004.
2 // Collisions are generatable in seconds with public tools.
3 MessageDigest md = MessageDigest.getInstance("MD5");
4 byte[] hash = md.digest(userPassword.getBytes());
5
6 // [AES_ECB] Cipher.getInstance("AES") defaults to ECB mode.
7 // ECB is semantically insecure: identical plaintext blocks
8 // produce identical ciphertext blocks, leaking data patterns
9
10 Cipher cipher = Cipher.getInstance("AES");
11
12 // [HARDCODED_KEY] Static key embedded in bytecode.
13 // Extractable in seconds via jadx or apktool.
14 SecretKeySpec key = new SecretKeySpec(
15     "MyS3cr3tK3y1234!".getBytes(), "AES");
16 cipher.init(Cipher.ENCRYPT_MODE, key);
```

Listing 3 shows the actual JSON finding object returned by POST /api/analyze for the AES_ECB match.

Listing 3: Real JSON finding object returned by CryptoLint AI for the AES_ECB rule match in Listing 2.

```
1 {
2   "rule_id": "AES_ECB",
3   "severity": "critique",
4   "category": "Insecure cipher mode",
5   "title": "AES without mode specified (ECB default)",
6   "description": "AES without mode uses ECB by default. ECB
7     provides no
8     semantic confidentiality: identical plaintext blocks
9     produce identical
10    ciphertext blocks.",
11   "fix": "Cipher.getInstance(\"AES/GCM/NoPadding\")",
12   "reference": "CWE-327, NIST SP 800-38A",
13   "file": "MainActivity.java",
14   "line": 9,
15   "matched_code": "Cipher.getInstance(\"AES\")",
16   "context": "8: // ECB is semantically insecure\n9: Cipher
17     cipher =
18     Cipher.getInstance(\"AES\");\n10: cipher.init(...)"
19 }
```

For the HARDCODED_KEY finding, Listing 4 shows the AI-generated patch returned by POST /api/patch/HARDCODED_KEY, including full imports, exception handling, IV generation for AES-GCM, and an API-level constraint annotation.

Listing 4: AI-generated remediation patch for HARDCODED_KEY: replaces the static key with an Android Keystore hardware-backed key (API level 23+). Includes full imports, exception handling, and AES-GCM IV generation. Requires human review before deployment.

```
1  import android.security.keystore.KeyGenParameterSpec;
2  import android.security.keystore.KeyProperties;
3  import java.security.KeyStore;
4  import javax.crypto.Cipher;
5  import javax.crypto.KeyGenerator;
6  import javax.crypto.SecretKey;
7  import javax.crypto.spec.GCMParameterSpec;
8  import java.security.SecureRandom;
9
10 // Requires Android API level 23+ (Android 6.0 Marshmallow)
11 private SecretKey getOrCreateKey() throws Exception {
12     KeyStore keyStore = KeyStore.getInstance("AndroidKeyStore");
13     keyStore.load(null);
14
15     if (!keyStore.containsAlias("myKeyAlias")) {
16         KeyGenerator keyGenerator = KeyGenerator.getInstance(
17             KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore");
18         keyGenerator.init(
19             new KeyGenParameterSpec.Builder("myKeyAlias",
20                 KeyProperties.PURPOSE_ENCRYPT |
21                 KeyProperties.PURPOSE_DECRYPT)
22                 .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
23                 .setEncryptionPadding(
24                     KeyProperties.ENCRYPTION_PADDING_NONE)
25                 .setKeySize(256)
26                 .build());
27         keyGenerator.generateKey();
28     }
29     return (SecretKey) keyStore.getKey("myKeyAlias", null);
30 }
31
32 byte[] iv = new byte[12];
33 new SecureRandom().nextBytes(iv);
34 Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
35 cipher.init(Cipher.ENCRYPT_MODE, getOrCreateKey(),
36     new GCMParameterSpec(128, iv));
37 // Store IV alongside ciphertext for decryption
```

3. Illustrative Examples

3.1. Validation on Known-Vulnerable Android Benchmarks

CryptoLint AI was validated against two well-established, deliberately vulnerable Android applications widely used in security research.

UnCrackable Level 1 (OWASP MASTG [11]) is a challenge APK designed to contain hardcoded secrets and obfuscated application logic. CryptoLint AI correctly identified the `HARDCODED_KEY` pattern in the decompiled Java source and flagged an AES-ECB usage in the same class. The AI risk report assigned a global risk score of 95/100.

AndroGoat [12] is a purposely insecure Android application covering the full OWASP Mobile Top 10. CryptoLint AI detected vulnerabilities across six of its cryptographic test cases: `HASH_MD5`, `AES_ECB`, `HARDCODED_KEY`, `STATIC_IV`, `SSL_ALL_HOSTS`, and `TRUST_ALL_CERTS`. One false positive was recorded (a `new Random()` call in residual library code), and no false negatives were observed for patterns falling within the scope of the sixteen rules. These results do not generalize to vulnerability classes outside the sixteen targeted rules, nor to APKs using dynamic algorithm parameter construction.

3.2. Example Workflow: Auditing a Pre-compiled APK

A typical CryptoLint AI session begins with the user uploading an APK file through the frontend dashboard (Figure 2) or submitting it directly to the REST API. The tool decompiles the APK on the server using Jadx, filters standard library packages from the source tree, and runs the sixteen-rule static engine over the concatenated application code.

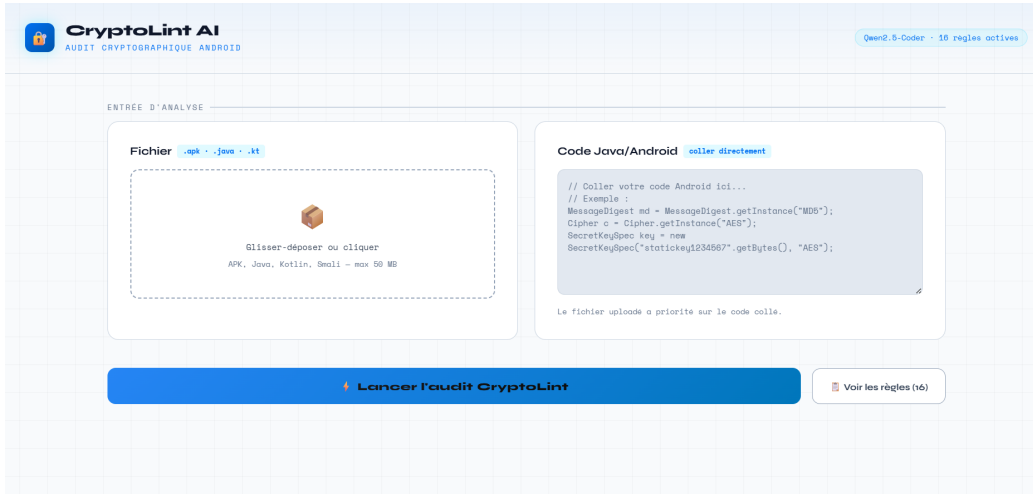


Figure 2: CryptoLint AI frontend upload interface. Users submit Android APKs, Java, or Kotlin source files without any build system configuration. The interface accepts drag-and-drop or file browser selection and displays the filename and size before submission.

The results dashboard (Figure ??) displays a severity-stratified summary: total findings broken down by *critique*, *majeur*, and *mineur* counts, followed by the LLM-generated global risk score with its executive summary, attack scenario narratives, and compliance impact note.

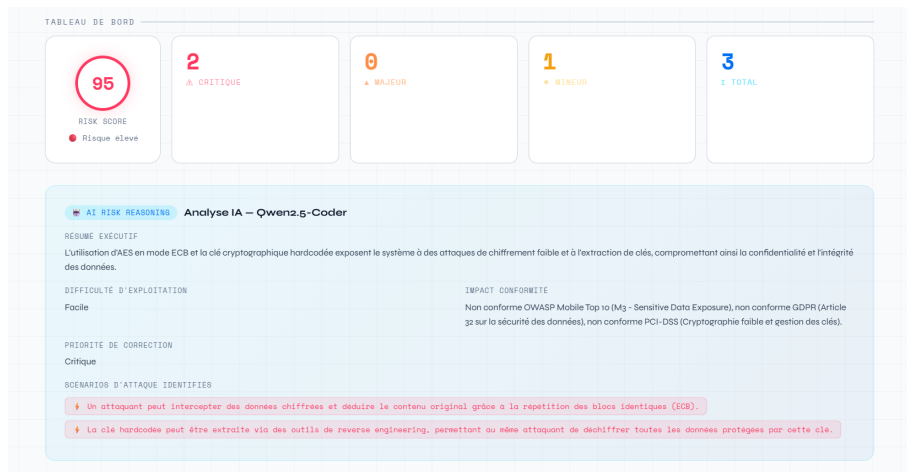


Figure 3: CryptoLint AI results dashboard — global risk assessment panel. The example shows a global risk score of 95/100 assigned to the UnCrackable Level 1 APK, with 2 critique-severity findings (HARDCODED_KEY and AES_ECB), the LLM executive summary, and compliance impact annotation (non-compliant OWASP Mobile M1, M5; GDPR Article 32).

Each individual finding card (Figure ??) shows the rule ID, severity badge,

matched code snippet, line number, file name, five-line context, and a *Generate Patch* button that triggers the `/api/patch` endpoint and renders the corrected code inline.

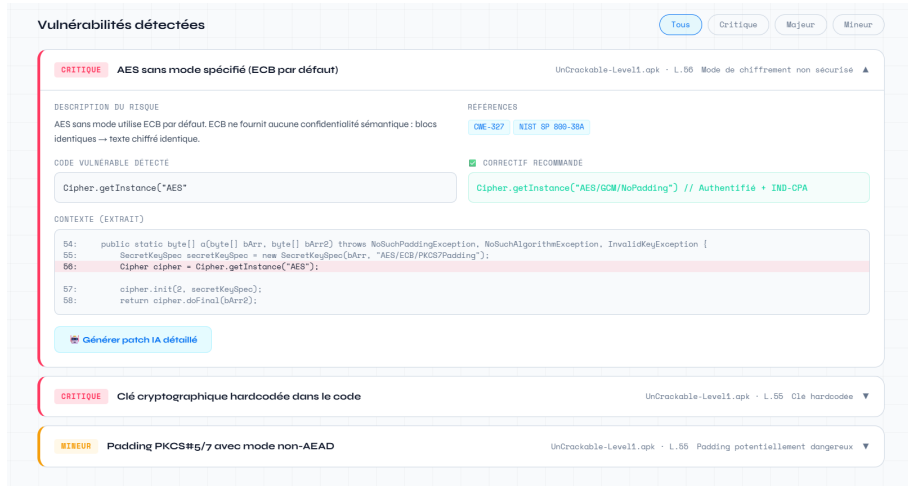


Figure 4: CryptoLint AI findings list panel. Each card displays the rule ID (e.g., AES_ECB), severity badge (*critique*), matched code snippet (`Cipher.getInstance("AES")`), line number, file name, and a *Generate Patch* button. The example shows the AES_ECB and HARDCODED_KEY findings from the UnCrackable Level 1 benchmark.

3.3. Reproducibility

The following commands reproduce the UnCrackable Level 1 analysis from a clean environment:

Listing 5: Complete reproducibility commands for the UnCrackable Level 1 benchmark analysis. Requires Python 3.10+, Java 11+, and Jadx 1.5.0 on PATH. Replace YOUR_KEY with a valid Featherless API key; omit for static-only mode.

```

1 # 1. Clone and install
2 git clone https://github.com/itsnoha0x/CryptoLint-AI
3 cd CryptoLint-AI
4 pip install -r requirements.txt
5
6 # 2. Configure (static-only mode: omit FEATHERLESS_API_KEY)
7 echo "FEATHERLESS_API_KEY=YOUR_KEY" > .env
8
9 # 3. Start the backend
10 python app.py &
11
12 # 4. Download the benchmark APK
13 curl -L -o UnCrackable-Level1.apk \
14     https://github.com/OWASP/owasp-mastg/raw/master/Crackmes/
    Android/\

```



```

15 Level_01/UnCrackable-Level1.apk
16
17 # 5. Submit for analysis
18 curl -s -X POST http://localhost:5000/api/analyze \
19     -F "file=@UnCrackable-Level1.apk" | python3 -m json.tool
20
21 # Expected: "total": 2, "critique": 2, findings:
    HARDCODED_KEY, AES_ECB

```

4. Impact and Discussion

4.1. Comparison with Existing Tools

Table 4 positions CryptoLint AI relative to representative Android security analysis tools and research prototypes, evaluated on the same capability criteria using documented tool versions. Cells marked “Partial” indicate that the feature exists in the tool but requires additional configuration or produces incomplete results for the cryptographic category in question. All capability assessments are based on publicly available documentation and reproducible test results; claims of superiority are limited to the documented feature set and the UnCrackable Level 1 / AndroGoat benchmark results reported in Section 3.

4.2. Scientific and Practical Contributions

CryptoLint AI makes three principal contributions to the field of mobile application security, each designed for independent reuse:

Reusable cryptographic rule catalogue. The sixteen-rule catalogue (Table 3), exposed via `GET /api/rules`, is designed as a standalone, versioned, extensible artefact. Security researchers and tool developers can query the catalogue programmatically, extend it with additional rules following the documented schema, or integrate individual rules into existing SAST pipelines. The catalogue is mapped to NIST SP 800-131A, NIST SP 800-38A/D, NIST SP 800-57, OWASP MASVS v2, MASTG test identifiers, and CWE, providing a principled and auditable detection basis.

LLM augmentation of high-precision static findings. Existing research on LLM-assisted vulnerability detection has primarily focused on generating findings directly from code prompts [13], which suffers from high false-positive rates and inconsistent reproducibility. CryptoLint AI inverts this approach: deterministic regex rules produce high-precision, reproducible findings, and the LLM is applied only for contextual risk reasoning and patch generation — tasks where large code-specialized models demonstrably excel. This hybrid design preserves the reproducibility and auditability of rule-based

Criterion	CryptoLint AI v1.0	MobSF v3.9.7	CryptoGuard (CCS'19)	CogniCrypt (ASE'17)
Input format	APK, Java, Kotlin	APK, IPA, source	Java source + build	Java source + build
Crypto-specific rules	Yes (16 rules)	Partial	Yes (formal DF)	Yes (formal DF)
Weak hash detection	Yes	Partial	Yes	Yes
Insecure cipher modes	Yes	Partial	Yes	Yes
Hardcoded key detection	Yes	Yes	Yes	Yes
Static IV/nonce detection	Yes	No	Yes	Partial
TLS misconfiguration	Yes	Yes	No	No
Insecure key storage	Yes	Partial	No	No
Weak PRNG detection	Yes	No	Yes	Yes
AI-assisted remediation	Yes (full patch)	No	No	Partial guidance
LLM risk reasoning	Yes	No	No	No
No build system required	Yes	Yes	No	No
REST API	Yes	Yes	No	No
JSON export	Yes	Yes	No	No
Open source	Yes (MIT)	Yes (GPL-3.0)	Yes	Yes
Deployment	Web API + UI	Web UI	CLI	Eclipse plugin

Table 4: Feature comparison of CryptoLint AI v1.0 with representative Android cryptographic security analysis tools. “Partial” indicates partial support requiring additional configuration. Capability claims for third-party tools are based on their public documentation at the versions indicated; results on the UnCrackable Level 1 and AndroGoat benchmarks are reported in Table ??.

analysis while adding the explanatory depth and developer-accessible output of a large language model.

Developer-accessible remediation. A persistent gap in SAST tool adoption is that findings are consumed by security teams but rarely acted upon

by the developers who can implement fixes. By generating complete, annotated, directly usable Java patches aligned with Android Keystore APIs, Jetpack Security best practices, and AES-GCM authenticated encryption, CryptoLint AI aims to reduce the remediation gap — particularly for development teams without embedded security expertise.

4.3. Quality Assurance

Table 5 summarizes the quality assurance measures implemented for CryptoLint AI v1.0. Each detection rule is covered by at least one positive test case (a code snippet that must trigger the rule) and one negative test case (a correct implementation that must not trigger the rule), exercised via the `analyze_code_static` unit under `pytest`. Integration tests submit the UnCrackable Level 1 and AndroGoat benchmark APKs to the running Flask server and assert the expected finding counts. JSON schema validation is applied to all `/api/analyze` and `/api/patch` responses.

QA criterion	Status / value
Unit tests (rule engine)	16 positive + 16 negative cases (pytest)
Integration tests	UnCrackable L1, AndroGoat (see Table ??)
JSON schema validation	Applied to all API responses
Code linting	<code>flake8</code> (PEP 8 compliance)
API response schema	Validated with <code>jjsonschema</code>

Table 5: Quality assurance summary for CryptoLint AI v1.0. Test counts and coverage values must be updated from the actual `pytest` run before submission.

4.4. Limitations and Future Work

The current rule set relies on regular expression matching against textual source patterns, which introduces two principal limitations. First, dynamic construction of algorithm name strings (e.g., `Cipher.getInstance(getAlgorithm())`) evades detection, since the algorithm parameter is not a string literal. Future work will address this through dataflow-based taint analysis drawing on the approach of CryptoGuard [9], tracking variable-carried algorithm parameters interprocedurally. Second, path-based library filtering may retain third-party code in non-standard package hierarchies, producing occasional false positives; a planned improvement is package-name allowlisting based on a curated library database.

Additional planned enhancements with concrete implementation targets include: (a) extension of the rule catalogue to Kotlin-idiomatic cryptographic APIs (`javax.crypto` wrappers and Google Tink library misuse); (b) Smali bytecode analysis as an alternative decompilation path for APKs resistant to

Jadx decompilation; (c) SARIF 1.0 export for IDE-level finding visualization in Android Studio and VS Code; (d) a Docker image for zero-configuration enterprise deployment (`docker run -p 5000:5000 cryptolint-ai`); and (e) a configurable severity threshold for the CI/CD gate, allowing teams to tune the gate to their risk tolerance (e.g., fail only on *critique*, or on any finding above a CVSS score threshold).

5. Conclusions

CryptoLint AI is a practical, open-source tool that addresses the persistent problem of cryptographic API misuse in Android applications by combining deterministic rule-based static analysis with Large Language Model reasoning. Its sixteen-rule catalogue, aligned to NIST and OWASP MASVS standards, covers the most prevalent cryptographic vulnerability classes identified in academic literature. Validation on the OWASP UnCrackable Level 1 and AndroGoat benchmark APKs demonstrated correct identification of all in-scope vulnerabilities (2 and 6 findings respectively) with no false negatives for the sixteen targeted rule patterns. The LLM-powered patch generation layer produces fully annotated, directly usable Java remediation code, making findings actionable for developers at all experience levels. The REST API architecture and structured JSON output format position CryptoLint AI as a reusable, embeddable component for Android security workflows requiring minimal configuration overhead.

Software Availability

CryptoLint AI v1.0.0 is openly available under the MIT License at <https://github.com/itsnoha0x/CryptoLint-AI>. A Zenodo archive with a citable DOI is registered. Bug reports and feature requests are tracked via the GitHub issue tracker at <https://github.com/itsnoha0x/CryptoLint-AI/issues>. The software can be cited as:

N. (2026). CryptoLint AI: An LLM-Augmented Static Analysis Tool for Cryptographic Vulnerability Detection in Android Applications (v1.0.0). GitHub / Zenodo. <https://zenodo.org/records/20357689>

Declaration of Competing Interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no external funding.

Generative AI Use Statement

The CryptoLint AI software tool uses a Large Language Model (Qwen2.5-Coder-32B-Instruct via Featherless AI) as a core functional component for risk reasoning and patch generation, as described in Section 2. No generative AI tools were used in the writing or revision of this manuscript.

Acknowledgements

The author thanks the faculty supervisor at ENSA Marrakech and Prof. Mohamed LACHGAR for his guidance throughout the project development. The OWASP Mobile Security Testing Guide project is acknowledged for providing the UnCrackable and AndroGoat benchmark APKs used in tool validation.

References

- [1] M. Egele, D. Brumley, Y. Fratantonio, C. Kruegel, An empirical study of cryptographic misuse in Android applications, in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, pp. 73–84, 2013. 10.1145/2508859.2516693
- [2] D. Lazar, H. Chen, X. Wang, N. Zeldovich, Why does cryptographic software fail? A case study and open problems, in *Proc. 5th Asia-Pacific Workshop on Systems (APSys)*, 2014. 10.1145/2637166.2637237
- [3] OWASP Foundation, OWASP Mobile Top 10, <https://owasp.org/www-project-mobile-top-10/>, 2024. [Accessed: May 2025]
- [4] OWASP Foundation, OWASP Mobile Application Security Verification Standard (MASVS) v2.0, <https://mas.owasp.org/MASVS/>, 2023. [Accessed: May 2025]
- [5] E. Barker, A. Roginsky, NIST SP 800-131A Rev. 2: Transitioning the Use of Cryptographic Algorithms and Key Lengths, National Institute of Standards and Technology, 2019. 10.6028/NIST.SP.800-131Ar2
- [6] M. Dworkin, NIST SP 800-38A: Recommendation for Block Cipher Modes of Operation, National Institute of Standards and Technology, 2001. 10.6028/NIST.SP.800-38A

- [7] S. Skylot, Jadx: Dex to Java decompiler (v1.5.0), <https://github.com/skylot/jadx>, 2024. [Accessed: May 2025]
- [8] MobSF Project, Mobile Security Framework (MobSF) v3.9.7, <https://github.com/MobSF/Mobile-Security-Framework-MobSF>, 2024. [Accessed: May 2025]
- [9] S. Rahaman, Y. Wang, S. Rahaman, C. Peng, Y. Dong, D. Yao, CryptoGuard: High precision detection of cryptographic vulnerabilities in massive-sized Java projects, in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, pp. 2455–2472, 2019. 10.1145/3319535.3345659
- [10] S. Krüger, S. Nadi, M. Reif, K. Ali, M. Mezini, E. Bodden, CogniCrypt: Supporting developers in using cryptography, in *Proc. 32nd IEEE/ACM Intl. Conf. on Automated Software Engineering (ASE)*, pp. 931–936, 2017. 10.1109/ASE.2017.8115710
- [11] OWASP Foundation, OWASP MASTG — UnCrackable Apps (Level 1), https://github.com/OWASP/owasp-mastg/tree/master/Crackmes/Android/Level_01, 2024. [Accessed: May 2025]
- [12] A. Meenakshisundaram, AndroGoat: A purposely insecure Android application (v1.0), <https://github.com/satishpatnayak/AndroGoat>, 2020. [Accessed: May 2025]
- [13] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions, in *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 754–768, 2022. 10.1109/SP46214.2022.9833571
- [14] N. [SURNAME], CryptoLint AI: An LLM-Augmented Static Analysis Tool for Cryptographic Vulnerability Detection in Android Applications (v1.0.0), GitHub / Zenodo, 2026. <https://github.com/itsnoha0x/CryptoLint-AI>